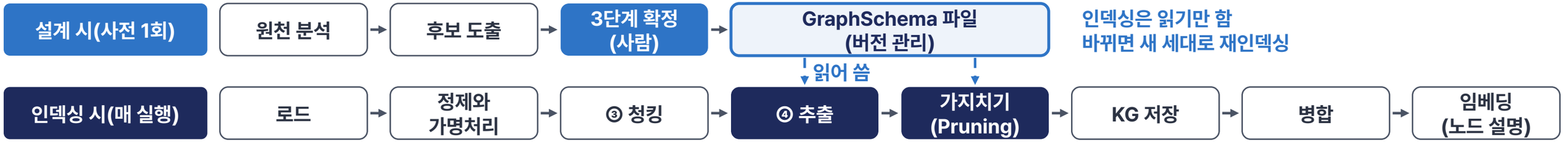
1. 스키마 도출: 노드 타입과 관계는 추출 전에 먼저 정함

스키마는 추출 결과가 아니라 추출기에 주는 허용 목록 — 먼저 정하지 않으면 같은 개념이 여러 라벨로 쪼개짐



스키마 없이 추출하면 (실측)

로컬 gemma-4 12B로 청크 2개를 스키마 없이 추출 → 라벨 11종

Brand · CardProduct = 카드 하나가 라벨 2개로

Fee · AnnualFee = 요금도 라벨 2개로

Channel · Benefit · Guardian · Grade · Date

Store · PerformanceCondition

GraphRAG 도구 기본 노드 타입/관계가 없거나 너무 일반적임

- Neo4j: 스키마를 안 주면 매번 입력 글에서 자동 추출(사람 검토 없음)
- MS GraphRAG 기본 타입: organization, person, geo, event
- LangChain+Neo4j: 허용 목록이 비면 아무 타입이나 만듦

스키마를 주면 (Native GraphSchema)

```
schema = GraphSchema.model_validate({
    "node_types": ["Card", "Benefit", "Fee"],
    "relationship_types": ["HAS_BENEFIT", "HAS_FEE"],
    "patterns": [("Card", "HAS_BENEFIT", "Benefit"),
                 ("Card", "HAS_FEE", "Fee")]})
graph = await extractor.run(chunks=chunks, schema=schema)
result = await GraphPruning().run(graph, schema)
```

① 추출 안내
허용 목록이 추출 프롬프트에 들어가 LLM이 그 안에서 고름

② 가지치기
목록 밖 노드와 관계를 지우고 지운 개수를 통계로 남김

실행 확인: 노드 타입을 주면 목록 밖 타입은 기본 불허

결론: 스키마는 인덱싱 전 설계 단계에서 확정해 파일로 고정, 인덱싱은 읽어 쓰기만 함.
정형 테이블은 변환 규칙으로, 문서는 표본 → 후보 → 사람 확정으로 스키마 설계

무단전재 및 배포 금지

1. 스키마 도출: 스키마에 정의하는 7가지

노드 타입과 관계, 각각의 속성, 연결 패턴이 뼈대 — 제약과 허용 범위로 무엇을 지킬지 정함

뼈대 ① ~ ⑤ 왼쪽부터 정의할 것, GraphSchema 칸, 카드 예

①	**노드 타입**	node_types[].label	**Card, Benefit**
②	**노드 속성**	node_types[] .properties	**Card {name, annual_fee}**
③	**관계**	relationship_types[] .label	**HAS_BENEFIT**
④	**관계 속성**	relationship_types[] .properties	**HAS_BENEFIT {valid_until}**
⑤	**연결 패턴**	patterns (출발, 관계, 도착)	**(Card, HAS_BENEFIT, Benefit)**

지킬 규칙 ⑥ ⑦

⑥	**제약**	constraints UNIQUENESS, EXISTENCE, KEY	**Card.name = KEY**
⑦	**허용 범위**	additional_node_types, additional_relationship_types, additional_patterns	**①~⑤에 지정된 목록 밖을 허용할지 여부**



7칸을 채운 Native GraphSchema — 실행 확인: name 없는 Card는 가지치기가 지움

```
schema = GraphSchema.model_validate({
  "node_types": [                          # ① ②
    {"label": "Card", "properties": [
       {"name": "name", "type": "STRING"},
       {"name": "annual_fee", "type": "INTEGER"}]},
    {"label": "Benefit", "properties": [...]}],
  "relationship_types": [{"label": "HAS_BENEFIT",   # ③ ④
     "properties": [{"name": "valid_until", "type": "DATE"}]}],
  "patterns": [("Card", "HAS_BENEFIT", "Benefit")],   # ⑤
  "constraints": [{"type": "KEY", "node_type": "Card",  # ⑥
                   "property_names": ["name"]}],
  "additional_node_types": False})              # ⑦
```

※ 최소한 = ①, ③, ⑤ + ⑥ 고유키. 속성은 질문에 쓰는 것만 정의하고, 타입마다 description을 적으면 LLM이 고르는 기준이 됨
※ 제약: 지정한 속성에 대해 UNIQUENESS는 고유 강제, EXISTENCE는 존재 강제, KEY는 고유 + 존재를 강제함. UNIQUENESS는 지정한 속성이 비어 있어도 허용하나 KEY는 불허함

무단전재 및 배포 금지

1. 스키마 도출: 정형 테이블 → 노드 타입과 관계 추출 규칙

모델 없이 규칙으로 바꿈 — 단, FK를 세기 전에 '그 표 자체가 조회 대상인가'부터 물음

테이블 1개

Q1. 그 표 자체가 조회 대상인 엔티티인가?

예

① 노드 타입
열 = 노드 속성
FK = 관계
FK 열은 바꾼 뒤 지움

FK가 0개 또는 1개인
보통 테이블은 대부분 여기로 옴

아니오

Q2. FK 몇 개?

② 조인 테이블: 2개이고 업무 열 없음
관계 1개로 흡수

③ 조인 테이블: 2개 + 업무 속성
관계 + 관계 속성
(조회하거나 집계하면 노드)

④ 3개 이상
노드 타입(중간 노드)

왜 Q1이 먼저? FK 수만 보면 이체 기록(FK 2개)이 관계로 흡수돼 이체 자체를 찾을 수 없음 (예 ①)

① 이체(출금계좌FK, 입금계좌FK, 금액, 일시) → **:계좌** —:출금→ **:이체** —:입금→ **:계좌**
조회 대상 → 노드 타입(FK가 2개여도)

② 사용자, 역할, 사용자역할(사용자FK, 역할FK) → **:사용자** —:맡음→ **:역할**
업무 열 없음 → 관계로 흡수(사용자역할 노드 없음)

③ 주문상품(주문FK, 상품FK, 수량, 단가) → **:주문** —:포함 {수량, 단가}→ **:상품**
업무 속성 → 관계 속성(판매 집계가 잦으면 노드)

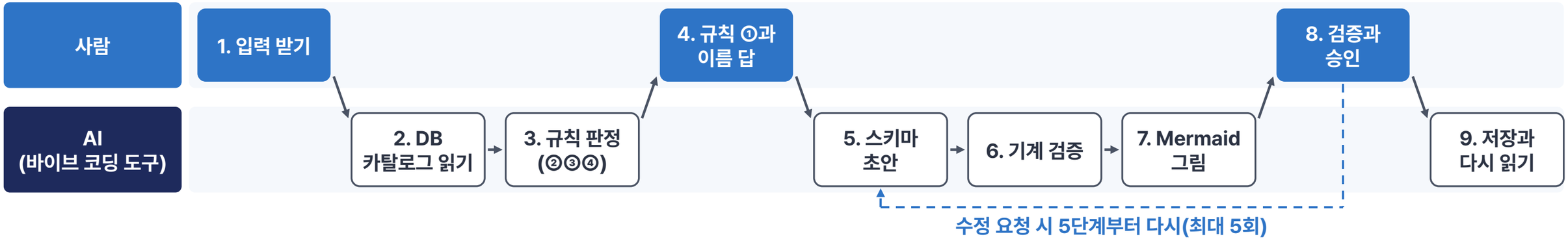
④ 수강신청(학생FK, 과목FK, 학기FK) → **:학생** —:신청→ **:수강신청** —:대상→ **:과목**
FK 3개 → 중간 노드(학기도 같은 방식), 홉 +1

Neo4j 공식 변환 규칙과 같음: 테이블 → 라벨 , FK → 관계 , 조인 테이블 → 관계 또는 관계 + 속성, 셋 이상 → 중간 노드
이름에 공백 없음(Neo4j 권고: 라벨은 PascalCase, 관계는 대문자_밑줄). 정형에서 나온 타입은 LLM 추출 비용이 들지 않음

무단전재 및 배포 금지

1. 스키마 도출: 프롬프트로 정형 테이블의 스키마 후보 도출

변환 규칙을 프롬프트에 담아 바이브 코딩 도구에 맡김 — AI는 읽고 판정하고 그리며, 사람은 세 번 답하고 승인함



사람이 답하는 곳 3번

1 출력 폴더와 DB 접속 정보
비밀번호는 .env에 직접 등록

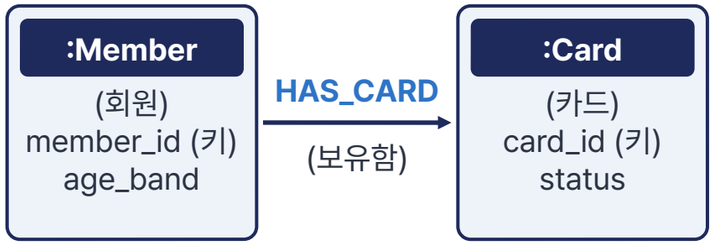
4 규칙 ① 후보(FK 2개 + 업무 열)를 노드 타입
또는 관계 + 관계 속성으로 정함
뺄 테이블, 이름 후보 승인

8 Mermaid 그림과 판정표를 보고 승인
(승인자와 일시를 기록)

AI가 지키는 규칙

- FK 개수는 열 수가 아니라 제약 수로 셈
- DB는 읽기 전용, 비밀번호는 .env에만 둠
- 라벨은 영문, 한글 이름은 description에 넣음
- 그림은 Mermaid만 씀
- 규칙 ① 판정과 이름은 추측해 확정하지 않음
- 후보 표시가 남으면 스키마 파일을 만들지 않음

승인 화면 예 (Mermaid 그림)



영문 라벨 아래에 한글 이름을 함께 적어
검토하는 사람이 바로 읽게 함

결과: graph_schema.json(승인본)과 approval.json(승인자, 승인 일시, 입력 파일 SHA-256). 인덱싱은 이 파일을 읽기만 함
GraphSchema 검증, 저장, 다시 읽기는 Neo4j 서버 없이 됨(실행 확인)

※ DB 카탈로그를 읽을 스키마는 원본 테이블이 있는 스키마로 지정하고, 서비스용 뷰(View)가 정의된 스키마(예: app)는 쓰지 않음

무단전재 및 배포 금지

1. 스키마 도출: 소스 문서 → 노드 타입과 관계 추출 방안

역량 질문(KG가 답해야 할 질문)을 주고 LLM으로 스키마 후보를 넓게 뽑은 뒤, 사람이 확정하고 표본 문서로 추출해 검증하여 GraphSchema로 고정함

① 역량 질문 정하기 (LLM 추천+사람확정) → **② 스키마 후보 넓게 뽑기 (LLM)** → **③ 스키마 후보 확정 (LLM 추천 + 사람확정)** → **④ 채우기 시험 (LLM + 사람)** → **⑤ 스키마 설계 확정 (코드)**

④ → ③ : 설명 고쳐 다시

- 문서 종류마다 답해야 할 질문 정의
- 질문에는 답이 될 노드타입이나 관계를 미리 쓰지 않음
- Ontology 101의 역량질문 가이드에 기반하여 LLM에 초안 작성 요청하고 사람 검토

- 문서 종류별로 1차로 묶고 필요시 하위 그룹별로 묶은 후 각 묶음마다 표본문서 추출: 시작 10건, 상한 30건(비율이 아니라 건수)
- SchemaFromTextExtractor에 질문을 examples로 넣음
- 앞서 나온 이름을 다시 쓰라는 안내는 넣지 않음

- ②의 후보목록과 역량질문을 주고 LLM에 스키마 초안 요청
- 초안: 추가, 제거, 병합할 것과 각각의 근거, 그리고 구체적 설명(해당과 비해당)
- 사람이 검토해 스키마 초안 확정(빈도로 거르지 않고 드문 이름도 살림)
- 사람이 값 규칙(중복제거, 필수존재) 정함

- 스키마 초안을 schema로 주고 추출한 뒤 GraphPruning으로 정리
- 표본 문서의 기대 답과 비교 (답이 없어야 하는 문서 포함)
- 잘못 뽑기가 많으면 설명을 고쳐 ③으로 돌아가 반복 수행

- GraphSchema.save()
- 스키마 설계 밖 노드타입, 관계, 속성은 만들지 않음
- 인덱싱은 이 파일을 읽기만 함
- 바뀌면 새 세대로 다시 인덱싱

주의

! ②에서는 앞 결과의 이름을 넘기지 않음. 넘기면 이름은 모이지만 새 개념이 안 나옴→ ④에서만 사용

! 대화 속 행위나 조항 규칙 같은 구조는 LLM이 잘 못 뽑음 → 사람이 직접 노드타입, 관계, 속성 넣기

! 여러 문서에 걸친 관계(카드 비교 등)는 청크 단위 추출로 나오지 않음 → 검색에서 처리

! 멈춤: 2건 이상 나온 새 이름이 연속 3건 없으면 그 층은 멈춤

출처: Neo4j GraphRAG Python 사용자 안내, Neo4j LLM Graph Builder 기능, Noy와 McGuinness, Ontology 101(역량 질문), Guest 외(2020) 포화 기준

무단전재 및 배포 금지

1. 스키마 도출: 단계 ① 역량 질문 정하기(예시: 상담 이력)

실제 사용자 질문을 모아 값을 빈칸으로 바꾸고 유형으로 묶은 뒤, 유형마다 역량 질문을 씀

실제 사용자 질문(출처)	값을 빈칸으로 → 질문 유형	역량 질문(상담 이력 5개)
D3 안내문 검색 평가 이 고객의 최근 불편은 무엇이고 이전 상담과 무엇이 달라졌는가? **A** **D**	**A** **{회원}의 최근 불편과 그 원인은?**	**A** 1. 고객이 말한 불만 원인은 무엇인가?
D3 안내문 검색 평가 어떤 제안을 받아들이거나 거절했으며 고객이 밝힌 이유는 무엇인가? **B**	**B** **{회원}이 받은 제안과 수용 또는 거절, 이유는?**	**B** 2. 상담사가 어떤 제안을 했는가?
평가셋 Q15 M-1042 회원은 생활비를 이 카드로 옮기는 것을 왜 원하지 않았나요? **B**	**C** **{회원}은 어떤 조건이면 다시 보나?**	**B** 3. 고객은 그 제안을 수용했는가, 거절했는가, 보류했는가? 그 이유는?
평가셋 Q16 M-3001 회원은 … 어떤 조건이면 옮기겠다고 했나요? **C**	**D** **{회원}의 이 상담은 어느 상담과 이어지나?**	**C** 4. 고객은 어떤 조건이 되면 다시 검토하겠다고 했는가?
		D 5. 이 상담은 같은 고객의 어떤 이전 상담과 이어지는가?

답이 될 이름은 쓰지 않음
피함: "Complaint 노드의 cause는?" 권함: "고객이 말한 불만 원인은?"

다른 문서도 같은 방법(각 5개)
약관: 어떤 조건을 채우면 어떤 면제나 혜택을 받는가?
혜택: 혜택의 한도와 제외 대상은 무엇인가?

초안은 LLM이 사용자 질문을 보고 작성하고 사람이 검토해 확정함. 개수는 정해 두지 않고 질문 유형마다 1개 이상

근거: D3 상담 이력 안내문(검색 평가 질문), RAGAS 평가셋 Q15와 Q16, 역량 질문은 스키마 후보 실험에 쓴 문장 그대로

무단전재 및 배포 금지

1. 스키마 도출: 단계 ② 스키마 후보 넓게 뽑기(예시: 상담 이력)

묶음별 표본을 1건씩 SchemaFromTextExtractor에 넣고(역량 질문만 examples로), 나온 이름을 모두 모음

묶음과 표본 수

약관
10청크 → 묶음 1개
10건 전부 봄

상담
48건 → 고객 세그먼트 묶음 6개
시작 12건(묶음마다 2건), 상한 30건

혜택
160청크 → 묶음 4개(공통 안내 4, 참고 카드 7, 카드 개요 64, 혜택 상세 85)
시작 4, 7, 10, 10건, 묶음마다 상한 30건

실제 1건: C-20260701-016(타사 관리 절차 비교)

고객: 전환 신청은 아직 안 했어요. 매달 신청을 반복해야 하는 게 불만의 원인이에요.
상담사: 지금 카드를 유지하시면 자주 쓰시는 곳 할인을 늘릴 수 있는지 확인해 볼까요?
고객: 그건 다음 결제일이 지나고 다시 볼게요. …
고객: 2026-06-26 상담 C-20260626-011과 이어지는 기록으로 남겨 주세요.

↓ **SchemaFromTextExtractor(examples = 역량 질문)**

나온 노드 타입 6개

Customer **Transaction** **Request** **Record**
Proposal **Condition**

관계 7개: MAKES_REQUEST, PROVIDES_PROPOSAL, HAS_CONDITION, CONTINUES_WITH 등
파랑 = 업무 개념. Customer 속성으로 email, birthdate까지 제안 → 가명처리 대상이라 ③에서 버림

30건을 모은 결과

노드 타입 53개, 관계 106개

많이 나온 타입 이름

Customer 29, Request 20, Transaction 18, Consultation 12, TransactionSummary 10

드물게 나온 업무 개념(역량질문이 찾는 개념)

Issue 5, Complaint 2, Proposal 2, Condition 2, REJECTS_CHANGE 1

같은 30건, 앞 결과 이름을 넘기면
타입 이름은 10개로 모이지만 업무 개념 2 / 5

넘기지 않으면,
타입이름은 53개로 흩어지지만 업무 개념 5 / 5

드물게 나온 이름이 핵심일 수 있음 — 빈도로 거르지 말고 ③에서 역량 질문을 기준으로 판단

실측: 2026-10-18, Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0, 합성 상담 이력 v3, 상담 30건(세그먼트 균형 순서), 업무 개념 = 불만, 제안, 응답, 재검토 조건

무단전재 및 배포 금지

1. 스키마 도출: 단계 ③ 스키마 후보 확정(LLM 추천 + 사람 확정)

후보 목록과 역량 질문을 LLM에 주면 병합, 제거, 추가 초안과 근거를 추천하고, 사람이 검토해 스키마 초안을 확정함

LLM 추천(괄호 = 근거가 된 역량 질문 번호)

병합

Issue, Complaint → Complaint (1)
Proposal, SUGGESTS, MAKES_PROPOSAL → Proposal (2)
Condition → ReconsiderCondition (4)
Consultation, Conversation, Session, Record → Consultation (5)
FOLLOWS, CONTINUES_FROM, PREVIOUS_SESSION → FOLLOWS (5)

제거

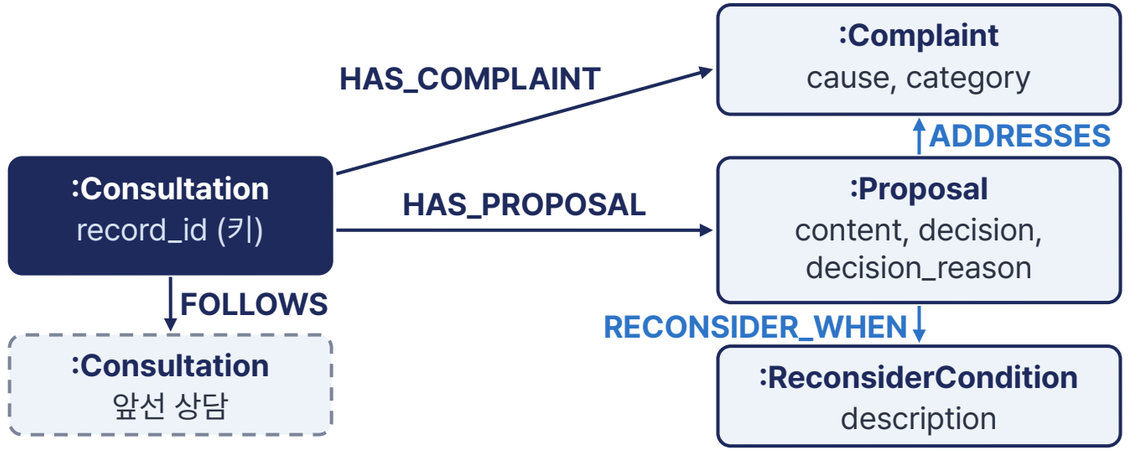
Request, Menu, Address, Order: 어느 질문에도 안 쓰임
Transaction, TransactionSummary: RDB에 이미 있는 집계
Customer의 name, email, birthdate: 가명처리 대상
Agent, Counselor: 메타데이터(상담사 가명ID)로 충분

추가

Proposal.decision, decision_reason (3): 후보는 REJECTS_CHANGE 1건뿐
ADDRESSES: 제안이 어느 불만을 겨냥했나 (1, 2)
RECONSIDER_WHEN: 보류한 제안의 재검토 조건 (4)

사람이 검토해 정한 것

- **고객 응답은 노드 대신 Proposal 속성(decision, decision_reason)으로**
- **상담 주제(Request)는 노드 대신 Consultation.topic 속성으로**
- **제약은 Consultation.record_id UNIQUENESS 하나(Community 판)**



Complaint 설명: 불만이나 불편을 분명히 말한 경우만. 문의 목적, 개인 사정은 아님

결과: 스키마 초안 = 노드 타입 4, 관계 5, 패턴 5, 제약 1. 사람은 질문과 데이터 사정(RDB, 가명처리, Neo4j 판)으로 판단

근거: 단계 ② 결과(이름 53개와 건수)와 역량 질문 5개, 초안 파일 skeleton_consult_v2.json(GraphSchema 검증 통과)

무단전재 및 배포 금지

1. 스키마 도출: 단계 ④ 채우기 시험(1) 실제 1건씩 대조

스키마 초안을 schema로 주고 상담 48건 전체를 실제 추출해, 강사용 정답표와 한 건씩 맞춰 봄

C-20260701-016, 불만이 있는 상담(v2 설명)

항목(역량 질문)	정답(대조표)	추출 결과	
불만 원인 질문 1	매달 신청을 반복해야 하는 게 불만의 원인	같음	맞음
재검토 조건 질문 4	다음 결제일이 지나고 다시 볼게요	같음	맞음
이전 상담 질문 5	C-20260626-011	같음(FOLLOWS 관계)	맞음
제안과 응답 질문 2, 3	할인을 늘릴 수 있는지 확인, 보류	자동납부 재등록 절차 비교, 수용	틀림

제안이 둘 이상인 상담에서 다른 안내를 고르는 실수가 남음(응답 엄격 일치 37 / 48의 이유)

C-20260310-018, 불만이 없는 주소 변경 문의

답이 없어야 하는 문서: 불만이 나오면 잘못 뽑은 것

v1 설명

Complaint {cause: "이사"}	틀림
Proposal 2개: 수령지 직접 변경 방법(수용), 후속 안내를 영업점 방식으로(수용)	틀림

v2 설명(좁힘)

Complaint 없음	맞음
Proposal 1개: 앱에서 본인 인증하고 수령지를 직접 바꾸는 방법, 수용, 이유 "집에서 바로 할 수 있는 쪽이 편해서요"	맞음

v1의 두 번째 제안은 절차 안내라 틀림. 정답표 제안은 1개

한 건씩 맞춰 보면 어떤 설명이 넓은지 보임. 답이 없어야 하는 문서가 있어야 잘못 뽑기를 찾음

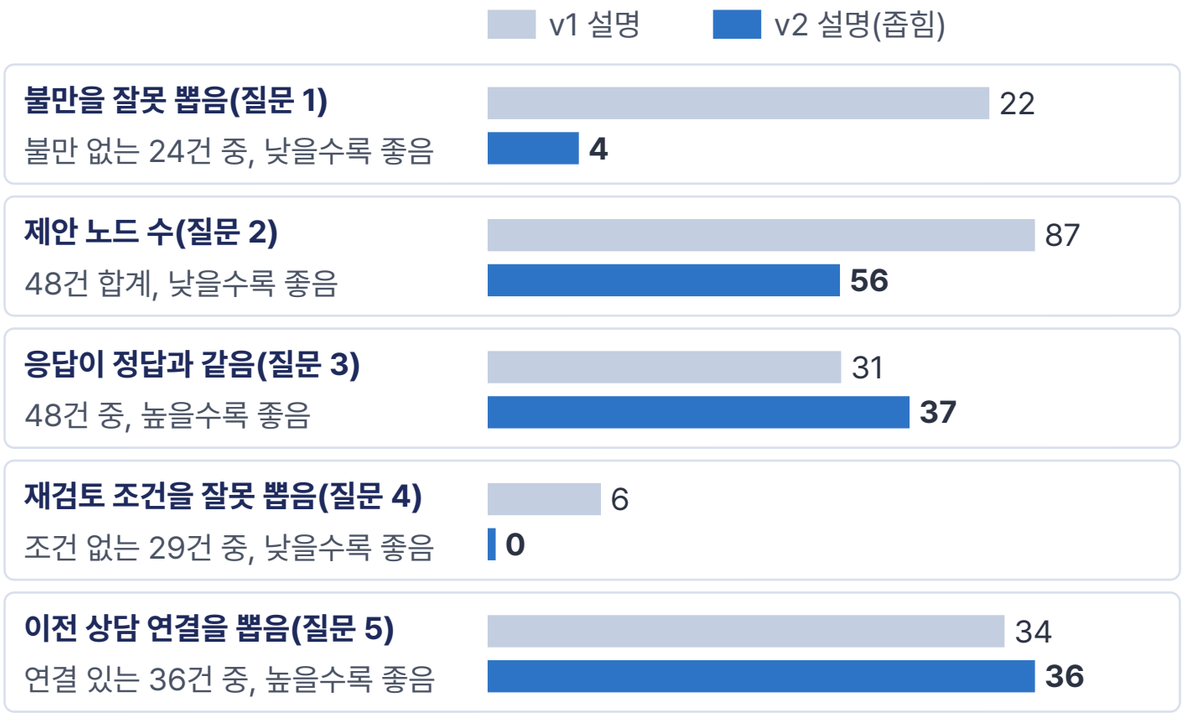
실측: 2026-10-18, Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0, 합성 상담 이력 v3, LLMEntityRelationExtractor와 GraphPruning, 정답: 강사용 상담근거 대조표(v3)

무단전재 및 배포 금지

1. 스키마 도출: 단계 ④ 채우기 시험(2) 전체 지표와 설명 고치기

48건 지표로 보고 잘못 뽑기가 많은 타입의 설명을 고쳐 ③으로 돌아감 — 한 번 고치자 불만 오탐이 22건에서 4건으로

상담 48건, 역량 질문 5개를 정답표와 비교



고친 설명(description)

Complaint

v1: 고객이 말한 불만과 그 원인

v2: 불만이나 불편을 분명히 말한 경우만. 단순 문의 목적, 개인 사정(이사, 지출 감소), 궁금증은 아님

Proposal

v1: 상담사가 한 제안과 고객의 응답

v2: 고객이 받아들이거나 거절할 수 있는 구체적 대안만. '확인해 드릴게요' 같은 절차 안내는 아님, 보통 1개

남은 문제

응답 엄격 일치 37 / 48: 제안이 여럿이면 다른 안내를 고름

불만 오탐 4건: '종이 영수증이 없어 불편'처럼 실제 불편 발화 → 정답 기준을 다시 볼 대상

결론: 이름보다 설명이 추출 품질을 좌우함. 지표가 기준에 닿을 때까지 ③과 ④를 되풀이

실측: 2026-10-18, Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0, 합성 상담 이력 v3, LLMEntityRelationExtractor와 GraphPruning, 정답: 강사용 상담근거 대조표(v3)

무단전재 및 배포 금지

1. 스키마 도출: 단계 ⑤ 고정과 그 뒤

시험을 통과한 스키마 초안을 GraphSchema.save()로 파일에 고정하고, 인덱싱은 그 파일만 읽음

저장과 다시 읽기(실측)

```
schema = GraphSchema.model_validate(초안)
schema.save("graph_schema_consult.json")
again = GraphSchema.from_file(
    "graph_schema_consult.json")
again == schema       # True
```

노드 타입 4 **관계 5** **패턴 5** **제약 1**

additional_* 3개 모두 False

Neo4j 서버 없이 됨. 상담 초안은 사람의 최종 승인 전이라 작업 폴더에서 시연함

고정의 뜻

- 목록 밖 노드 타입과 관계는 가지치기가 지움 (additional_* False)
- 스키마나 원천이 바뀌면 새 세대로 다시 인덱싱
- 실제 예: 상담 원문 v3를 반영할 때 새 세대 gen-d3v3-20261018을 만들고 상담 48청크만 새로 임베딩, 170청크는 재사용

인덱싱이 이 파일을 읽는 곳



추출 프롬프트의 허용 목록과 가지치기 기준으로 함께 씀

문서 종류별 진행

상담

① ~ ④ 완료, ⑤는 사람의 최종 승인 남음

혜택

②까지. 질문을 넣자 Exclusion 3 → 25건, 후보가 거의 그대로 초안

약관

②까지. 이름 76%가 1건에서만 나옴 → 조항 구조 초안을 사람이 먼저

결론: 다섯 단계는 문서와 상관없이 같고, 문서 성격에 따라 LLM 후보를 그대로 쓰는 비중과 사람 판단의 비중이 달라짐

실측: 2026-10-18, Groq openai/gpt-oss-120b, neo4j-graphrag 1.22.0, 합성 상담 이력 v3, 인덱서 세대 gen-d3v3-20261018(청크 218개 중 48개 새 임베딩)

무단전재 및 배포 금지